6 Docs to Write Before You Vibe Code Your App

Nov 3, 2026 · @Aditya Anand

How to use these six docs

Write the docs in the order below: each one feeds the next, and your AI agent builds from all six together.

1. Pick one app idea and write it in two lines.
2. Copy the template and the prompt under each doc into Claude or ChatGPT.
3. Paste the earlier docs into the chat before you generate the next one.
4. Edit every output yourself. Replace anything vague with a decision.

Use this opening for every prompt:

```
I'm building: [one-line app idea]
Audience: [who will use it]
Generate the [DOC NAME] using the template below.
Ask me questions first if anything is unclear.
Do not add features I did not mention.
```

1. PRD (Product Requirements Document)

The PRD says exactly what you are building: a short app overview, then every feature in detail.

Template:

```
# PRD: [App name]

## Overview
[2-3 lines: what the app does, who it is for, what problem it solves]

## Goals
-

## Out of scope (v1)
-

## Features
```

```
### Feature 1: [name]
- What it does:
- What the user does:
- What happens next:
- Edge cases:

### Feature 2: [name]
- What it does:
- What the user does:
- What happens next:
- Edge cases:
```

Prompt:

```
Write a PRD for my app using the template above.
List every feature in detail: what it does, what the user does, what happens next, and edge cases.
Mark anything that should wait for v2 under Out of scope.
```

2. TRD (Technical Requirements Document)

The TRD locks your technical decisions: which stack, which tools, and which APIs.

Template:

```
# TRD: [App name]

## Stack
- Frontend:
- Backend:
- Database:
- Hosting:

## Tools
- Authentication:
- Payments:
- Email / notifications:
- Analytics:

## APIs
| API | Used for | Auth method | Limits / cost |
| --- | --- | --- | --- |
|  |  |  |  |
```

```
## Constraints
- Performance:
- Security:
- Device / browser support:
```

Prompt:

```
Here is my PRD: [paste PRD]
Write a TRD using the template above.
Pick one option for every decision and give a one-line reason.
Keep the stack simple enough for a solo builder.
```

3. App Flow

The App Flow maps how a user moves through the app: every page, every journey, and what each button click does.

Template:

```
# App Flow: [App name]

## Pages
- Landing
- Sign up / Login
- Dashboard
- [Page]

## User journey
1. User lands on [Page A]
2. Clicks [button] -> goes to [Page B]
3. Does [action] -> sees [result] on [Page C]

## Button and action map
| Page | Element | On click | Next page / result |
| --- | --- | --- | --- |
|  |  |  |  |

## Error and empty states
- Wrong password:
- No data yet:
- Network failure:
```

Prompt:

```
Here are my PRD and TRD: [paste both]
Write the App Flow using the template above.
Cover every page, the full user journey, and what happens on every button
click.
```

4. UI-UX Design Brief

The Design Brief fixes the look and feel: color palette, typography, components, style, and a rough layout for each screen.

Template:

```
# UI-UX Design Brief: [App name]

## Style
- Mood (clean / playful / minimal / bold):
- Reference apps:

## Color palette
| Role | Hex |
| --- | --- |
| Primary |  |
| Secondary |  |
| Background |  |
| Text |  |
| Error / success |  |

## Typography
- Heading font and size:
- Body font and size:

## Components
- Buttons:
- Inputs:
- Cards:
- Modals:
- Navigation:

## Screens
### [Screen name]
- Layout, top to bottom:
- Key elements:
```

Prompt:

```
Here are my PRD and App Flow: [paste both]
Write a UI-UX Design Brief using the template above.
Give exact hex codes and font names.
Describe each screen from top to bottom.
```

5. Backend Schema

The Backend Schema defines how user data is stored and organized: the authentication flow, every table, its columns, and how the tables relate.

Template:

```
# Backend Schema: [App name]

## Authentication flow
1. Sign up:
2. Login:
3. Session / token:
4. Logout and password reset:

## Tables
### users
| Column | Type | Notes |
| --- | --- | --- |
| id | uuid | primary key |
| email | text | unique |
| created_at | timestamp |  |

### [next table]
| Column | Type | Notes |
| --- | --- | --- |
|  |  |  |

## Relationships
- users 1 -> many [table]

## Access rules
- Who can read and write each table:
```

Prompt:

```
Here are my PRD, TRD and App Flow: [paste all three]
Write the Backend Schema using the template above.
Include the authentication flow, every table with columns and types, and the
relationships between tables.
```

6. Implementation Plan

The Implementation Plan is the exact step-by-step build order, so the AI agent knows what to build first and what comes next.

Template:

```
# Implementation Plan: [App name]

Rules for the agent:
- Build one step at a time.
- After each step, run the app and confirm it works before moving on.

## Steps
1. Project setup with the stack from the TRD
2. Database and schema
3. Authentication
4. Core feature 1
5. Core feature 2
6. UI from the Design Brief
7. Testing and bug fixes
8. Deploy

## Done when
- Step 1:
- Step 2:
```

Prompt:

```
Here are all five documents: [paste PRD, TRD, App Flow, Design Brief, Backend
Schema]
Write an Implementation Plan using the template above.
Order the steps so each one can be tested before the next begins.
Add a "Done when" line to every step.
```

Hand the six docs to your AI agent

Save all six as files in a `docs` folder inside your project, then give the agent this first prompt before any build request.

```
Read all six documents in the docs folder before writing any code:
PRD, TRD, App Flow, UI-UX Design Brief, Backend Schema, Implementation Plan.
Follow the Implementation Plan one step at a time.
If anything is missing or two documents disagree, ask me before building.
```

When you change a feature, update the matching doc first, then ask the agent to build the change.